

Learning to Loading/Selecting LLM for your future assignments

Generative AI and LLMs

Prof G C Nandi

Full marks-10

Hugging face hub contains a huge number of language models (about 800000) as on January, 2026

We will use a lightweight Phi-3-mini LLM (details provided here:
<https://huggingface.co/microsoft/Phi-3-mini-4k-instruct>).

When you use Large Language Model two models are loaded :

1. The generative model itself
2. **Its underlying Tokenizer (A token is the basic unit an LLM reads and writes. It can be a whole word, part of word (subword), punctuation, or even a symbol Ex: **Unbelievable !** (un, believ, able, !), Word2vec- it is an early embedding method that converts word into dense vectors so that words with similar meanings have similar vectors (For example, the relationship king – man + woman = queen illustrates static embeddings(Examples: **Word2Vec, GloVe, FastText**). However, modern LLMs rely on dynamic contextual embeddings (Examples: **ELMo, BERT, GPT, Transformer-based LLMs**), which we will explore in detail when we dive deeper into the transformer model).**

The tokenizer is in charge of splitting the input text into tokens before feeding to the generative model

Requirements of your hardware/computational resources:

You will need an NVIDIA GPU or similar devices. If you do not have access to GPU, use free Google Colab notebooks

1. Your first task is to download a model by running the code (will take a few minutes).

```
from transformers import AutoModelForCausalLM, AutoTokenizer
```

```

# Load model and tokenizer
model = AutoModelForCausalLM.from_pretrained(
    "microsoft/Phi-3-mini-4k-instruct",
    device_map="cuda",
    torch_dtype="auto",
    trust_remote_code=True,
)
tokenizer = AutoTokenizer.from_pretrained("microsoft/Phi-3-mini-4k-instruct")

```

By running these codes you will be able to downloading the model

2. The model is now capable of generating text. However, you may simplify the process by encapsulating model, tokenizer, and text generation process into a single function, namely `transformer.pipeline`

```

from transformers import pipeline

# Create a pipeline
generator = pipeline(
    "text-generation",
    model=model,
    tokenizer=tokenizer,
    return_full_text=False,
    max_new_tokens=500,
    do_sample=False
)

```

Learn the following parameters:

- i) `return_full_text` (by setting this parameter to false, the prompt will not be returned but merely the output of the model).
- ii) `max-new_tokens` (the maximum number of tokens the model will generate. By setting a limit, we prevent long and unwieldy outputs some model might continue generating output until they reach their context window.
- iii) `do_sample`- by selecting this 'false' the model will select the next most probable token.

3. Generate any text of your choice (submit three outputs)